Simulating the Spread of the Ebola Virus using Cellular Automata in a Parallel Computing Environment

Project Plan

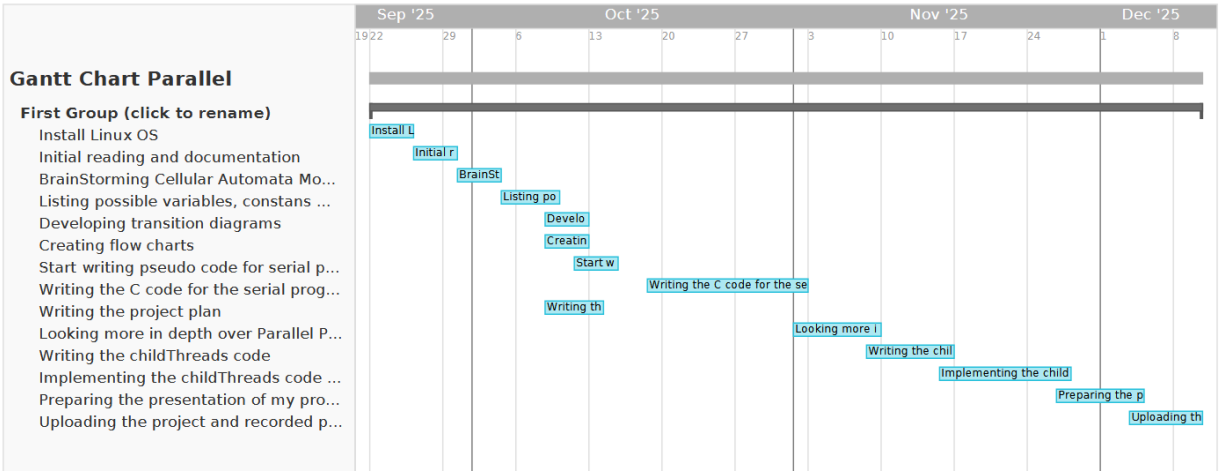
By Kevin Ceausu – B00128171

2025-10-17

Declaration of academic honesty

I am aware of the University policy on plagiarism in assignments and examinations (3AS08). I understand that plagiarism, collusion, and copying are grave and serious offences in the University, and I will accept the penalties that could be imposed if I engage in any such activity. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. I declare that this material, which I now submit for assessment, is entirely of my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work.

Gantt Chart with the planned phases of my project



Written description of the phases of the Gantt Chart

1. Installing Linux OS

This subject along with its project isn't designed to only teach us about parallel programming, but also to teach us how to adapt and work on a different operating systems instead of only being used to work on Windows. This is why, for this projects development we will be required to use Linux, so we first need to install it.

2. Initial Reading and Documentation

After successfully installing Linux on our personal computer, our first step for this project is to start reading the Project Description and Guidance document along with the paper "A Cellular Automaton Framework for Infectious Disease Spread Simulation" by Pfeifer et al. This will help us develop a more profound idea about what we need to do, what to achieve and how to do it with our project.

3. Brainstorming the Cellular Automata Model

After doing our initial documentation, we need to start deciding on our approach to the Cellular Automata Model. First we need to get a better understanding of SIR/D basic model and the SEIR/D latent model. We will need to brainstorm ideas about how we will start writing our pseudo-code, actual code, and any charts that come with them.

4. Listing possible variables, constants and macros

In this phase, we are still brainstorming about what exactly we need to have and do with our project, but this time we are doing it in more depth. We are beginning to understand the basics of the Cellular Automata Model and start listing possible variables, constants and macros that we may use in our code, such as:

Variables:

- int grid[ROWS][COLS]: This will be used to to create a 2d array to store the current state of the map.

- int rows : number of rows in the grid
- int cols: number of columns in the grid
- int generations: number of simulation steps to run
- int susceptible_count: count susceptible cells

Constants:

- const int S = 0 // constant for creating Susceptible cells
- const int I = 1 // constant for creating Infected Cells
- const int R = 2 // constant for creating Removed cells aka dead
-const int rows = 1000
- const int cols = 1000

Macros:

- #define IS_SUSCEPTIBLE(cell) (cell == S) // checks if a cell is susceptible
- #define IS_Infected(cell) (cell==I) // checks if a cell is infected
-#define IS_Dead(cell) (cell==R)// checks if a cell is dead

5. Developing transition diagrams

In this phase we are developing transition diagrams of the basic and latent model to develop a better idea of all transitions we will need to implement in our project. Those diagrams will help us create a better image of how the project should work.

6. Creating flow charts

The flow charts will be created in order to help us further plan our development of our code and project, and create a better idea of how exactly we will need to implement variables, constants and macros we previously discussed about.

7. Start writing pseudo code for serial program

In order for us to be able to develop our project from scratch and write our actual code, we will first need to write the pseudo code so we can provide a clear description of that steps that will be taken in the process of creating the project. Pseudo code is particularly useful because it provides a clear way of helping all programmers understand our steps regardless of the programming languages that they are or aren’t used to.

8. Writing the project plan

Writing the project plan is an important step of developing any project as it’s the tool used to provide us with a clear roadmap for how our project will be executed and controlled, ensuring we don’t fall behind any of our steps and manage to deliver everything on time.

9. Writing the C code for the serial program

This is the part where we actually start developing our project, using the C programming language to write the code in order to have the serial program.

10. Looking more in depth over Parallel Programming and Pthreads

As this is our first time working with Pthreads, we will be required to do more research on how exactly Pthreads work and how they are used in different tasks. By doing this research we will ensure that we have the necessary understanding in order to begin developing our Parallel Program.

11. Writing the Pthreads code

After doing our research, we will start writing our childThreads code that will allow us to use multithreading for our program.

12. Implementing the Pthreads code into my serial program to make it parallel

With the previous step finished, we can begin working on our parallel program by implementing the Pthreads we have previously worked on to our serial program. This should help our program execute at a faster speed.

13. Preparing the presentation of my project

After managing to conclude my development of the parallel program, I can begin working on the presentation of my project, as all students will need to have a recorded presentation to their project explaining what it does and how it works.

14. Uploading the project and recorded presentation

This is the last step of the project, where everything we have discussed in our project plan has been achieved, and the final program is ready to be uploaded and graded.